МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №3

"Ітератори, генератори та потокова обробка даних"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

проф. Свелеба С. А.

Львів-2026

Мета роботи: Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, інструкціями yield та yield from, генераторними виразами (generator expressions), модулем itertools, принципом лінивих обчислень (lazy evaluation) та потоковою обробкою великих наборів даних без повного завантаження у пам'ять. Індивідуальний варіант 1: потокова обробка даних системи обліку студентів.

Теоретичні відомості:

Ітераційна модель Python базується на протоколі ітератора, де об'єкт iterable реалізує метод __iter__(), а об'єкт iterator реалізує методи __iter__() та __next__(). Генераторні функції з ключовим словом yield дозволяють призупиняти виконання зі збереженням стану локальних змінних, забезпечуючи ліниві обчислення за вимогою споживача. Конструкція yield from делегує генерацію вкладеному ітератору. Модуль itertools надає високоефективні інструменти (islice, chain, pairwise, count, groupby) для побудови швидких конвеєрів. Лінива потокова обробка (streaming) дозволяє оперувати масивами даних практично необмеженого розміру за постійного константного використання оперативної пам'яті O(1).

Виконання роботи

1. Постановка індивідуального завдання (Варіант 1: Потокова обробка даних системи обліку студентів). Канонічна схема вхідних даних CSV включає 5 полів: student_id,first_name,last_name,group,average_grade. Відповідно до варіанта розробив та верифікував повноцінний потоковий конвеєр, що включає: потокове читання файлу CSV з підтримкою кодування UTF-8-SIG (BOM) та валідацію записів студентів, фільтрацію за академічною групою та мінімальним середнім балом, пошук студентів за прізвищем або ID із раннім виходом (early termination), однопрохідний розрахунок потокового середнього бала та min/max average_grade без повної матеріалізації вибірки, визначення перших N студентів із найвищим балом (TRUE Top-N heap ranking на основі heapq.nlargest) на відміну від перших N у порядку файлу (first_n через islice), пакетну обробку записів студентів (batch processing), підрахунок кількості студентів за групами, а також відстеження кількості пропущених некоректних рядків.

2. Опис синтетичного набору даних (Synthetic Dataset Disclosure). Для забезпечення відтворюваності та об'єктивного порівняння eager та lazy конвеєрів у модулі streaming_experiments.py реалізував функцію генерації синтетичних CSV-файлів пакетами по 50 000 рядків із детермінованим генератором random.Random(42). Використано фіксовані набори типових українських імен (Олександр, Андрій, Максим, Іван, Тарас, Ольга, Олена, Марія, Ірина, Леся), прізвищ (Шевченко, Франко, Українка, Мельник, Коваль, Бондар, Бойко, Кравченко, Ткаченко, Поліщук) та академічних груп (ФеП-31,

ФеП-32, ФеП-33, ФеП-34). Середній бал генерується рівномірно в діапазоні від 50.0 до 100.0 з округленням до двох знаків. Для демонстрації граничних випадків та некоректних даних підготував тестовий файл demo_students.csv (27 рядків), який містить 25 валіplacених студентів, рядок із порожньою групою (ID 26) та рядок із оцінкою 150.0 (ID 27), що виходить за межі [0, 100].

3. Структура проєкту та розподіл обов'язків між модулями. Проєкт побудовано за схемою src-layout у пакеті student_manager:
- models.py: датаклас Student із валідацією полів та підтримкою student_id;
- streaming.py: повнофункціональний модуль потокової обробки (генератори читання, валідації, фільтрації, трансформації, batching, агрегації, власний StudentIterator);
- lab3_demo.py: консольний сценарій комплексної демонстрації всіх операцій варіанта 1;
- streaming_experiments.py: бенчмарк-модуль для замірів elapsed time, peak memory (tracemalloc), time-to-first-result на наборах 10k, 100k, 500k та 1M записів;
- tests/test_streaming.py та tests/test_streaming_experiments.py: автоматичні тести коректності конвеєра, лінивості та ізоляції ресурсів.

```
PS C:\Users\ponti> Get-ChildItem -Path src\student_manager\streaming*, src\student_manager\lab3_demo* -File
| Select-Object -ExpandProperty Name
streaming.py
streaming_experiments.py
lab3_demo.py
PS C:\Users\ponti> _
```

4. Розширення моделі даних Student (models.py). Датаклас Student підтримує як позиційне створення без ID для зворотної сумісності з Лабораторними 1-2, так і роботу з числовим student_id (int > 0). Валідація контролює непустий стан текстових полів та діапазон середнього бала від 0.0 до 100.0.

```python
from dataclasses import dataclass


@dataclass
class Student:
    first_name: str
    last_name: str
    group: str
    average_grade: float
    student_id: int | None = None

    def __post_init__(self) -> None:
        self.first_name = self.first_name.strip()
        self.last_name = self.last_name.strip()
        self.group = self.group.strip()
        if not self.first_name or not self.last_name or not self.group:
            raise ValueError("Name, surname and group cannot be empty")
        if not 0 <= self.average_grade <= 100:
            raise ValueError("Average grade must be between 0 and 100")
        if self.student_id is not None:
            # перевіряємо шоб айді був цілим додатнім числом і не bool
            if isinstance(self.student_id, bool) or not isinstance(
                self.student_id, int
            ):
                raise TypeError("Student ID must be an integer")
            if self.student_id <= 0:
                raise ValueError("Student ID must be a positive integer")

    @property
    def full_name(self) -> str:
        return f"{self.first_name} {self.last_name}"
```

5. Архітектурна схема потокового конвеєра (Pipeline Stages). Конвеєр складається з ланцюжка зв'язаних лінивих генераторів, де кожен етап отримує ітероване джерело (Iterable) і повертає ітератор (Iterator), передаючи об'єкти по одному за запитом кінцевого споживача (Pull-модель). У таблиці нижче наведено повний перелік фактичних етапів конвеєра варіанта 1.

Етап конвеєра	Функція / Модуль	Тип операції	Опис етапу в архітектурі
1. CSV джерело	data/input.csv (demo_students.csv)	Вхідний файл	Зовнішній CSV-файл із підтримкою кодування UTF-8-SIG (BOM) та канонічними полями.
2. Читання рядків	read_lines(path)	Lazy source	Потокове читання файлу по рядках через yield from file без завантаження всього файлу в RAM.
3. Сирий CSV парсинг	parse_csv_raw(lines)	Lazy parsing	Використання csv.reader(lines) та yield from reader для коректної обробки лапок і спецсимволів.
4. Очищення та заголовок	clean_rows(rows)	Lazy cleaning	Валідація обов'язкового заголовка PRESCRIBED_HEADERS та очищення пробілів у полях рядків.
5. Парсинг і валідація	parse_students(rows, stats)	Lazy validation	Перетворення рядків у датаклас Student, пропуск битих рядків та підрахунок помилок у StreamStats.
6. Лінива фільтрація	filter_by_group, filter_by_min_grade	Lazy filtering	Генераторні функції з yield для фільтрації за академічною групою та мінімальним середнім балом.

7. Трансформація кортежів	transform_to_summary(students)	Lazy mapping	Генератор із yield для перетворення повнорозмірних Student у компактні кортежі зведеного звіту.
8. Пакетування та статистика	batch_students, calculate_statistics	Terminal sink / batch	Пакетне групування через islice або однопрохідний підрахунок середнього, min/max та груп.

6. Етапи читання, сирого парсингу та очищення CSV (streaming.py). Функція read_lines відкриває файл із підтримкою utf-8-sig і повертає рядки через yield from file. Функція parse_csv_raw делегує розбір бібліотечному csv.reader через yield from reader, що запобігає ламанню коми в лапках. Функція clean_rows зчитує заголовок, перевіряє його строгий збіг із PRESCRIBED_HEADERS та ліниво повертає очищені від пробілів рядки.

```python
PRESCRIBED_HEADERS: tuple[str, ...] = (
    "student_id",
    "first_name",
    "last_name",
    "group",
    "average_grade",
)


@dataclass
class StreamStats:
    invalid_records: int = 0


def read_lines(path: str | Path) -> Generator[str, None, None]:
    # читаєм файл потоково з підтримкою bom через utf-8-sig
    with Path(path).open("r", encoding="utf-8-sig", newline="") as file:
        yield from file


def parse_csv_raw(lines: Iterable[str]) -> Iterator[list[str]]:
    # парсимо csv стандартним рідером шоб не ламати переноси у лапках
    reader = csv.reader(lines)
    yield from reader


def clean_rows(rows: Iterable[Sequence[str]]) -> Iterator[list[str]]:
    # зачищаєм пробіли та перевіряємо точний заголовок
    it = iter(rows)
    try:
        header_row = next(it)
    except StopIteration:
        raise ValueError("CSV is empty, missing header")

    cleaned_header = [field.strip() for field in header_row]
    if tuple(cleaned_header) != PRESCRIBED_HEADERS:
        raise ValueError(
            f"Invalid CSV header: expected {list(PRESCRIBED_HEADERS)}, got {cleaned_header}"
        )

    for row in it:
        yield [field.strip() for field in row]
```

7. Етап валідації записів студентів та керування життєвим циклом файлу. Функція parse_students перевіряє довжину рядка, типи та діапазони значень. Некоректні рядки відкидаються, а лічильник помилок фіксується в екземплярі StreamStats або invalid_counter. Контекстний менеджер open_student_stream обгортає потік читання за допомогою closing(lines) та викликає gen.close() у блоці finally, що гарантує негайне закриття дескриптора файлу ОС навіть при достроковому перериванні споживання (ранньому виході).

```python
def parse_students(
    rows: Iterable[Sequence[str]],
    *,
    stats: StreamStats | None = None,
    invalid_counter: list[int] | None = None,
) -> Generator[Student, None, None]:
    # валідуємо поля студента і рахуємо пропущені некоректні рядки
    for row in rows:
        if len(row) != 5:
            if stats is not None:
                stats.invalid_records += 1
            if invalid_counter is not None:
                invalid_counter[0] += 1
            continue

        id_str, first_name, last_name, group, grade_str = (
            row[0].strip(),
            row[1].strip(),
            row[2].strip(),
            row[3].strip(),
            row[4].strip(),
        )

        try:
            student_id = int(id_str)
            if student_id <= 0:
                raise ValueError("ID must be positive")
            if not first_name or not last_name or not group:
                raise ValueError("Text fields cannot be empty")
            grade = float(grade_str)
            if math.isnan(grade) or math.isinf(grade) or not (0 <= grade <= 100):
                raise ValueError("Grade out of range or nan/inf")
            student = Student(
                first_name=first_name,
                last_name=last_name,
                group=group,
                average_grade=grade,
                student_id=student_id,
            )
        except (ValueError, TypeError):
            if stats is not None:
                stats.invalid_records += 1
            if invalid_counter is not None:
                invalid_counter[0] += 1
            continue

        yield student
```

```python
@contextmanager
def open_student_stream(
    path: str | Path, *, stats: StreamStats | None = None
) -> Iterator[Iterator[Student]]:
    # контекстний менеджер для коректного закриття файлу після стріму
    lines = read_lines(path)
    with closing(lines):
        gen = parse_students(clean_rows(parse_csv_raw(lines)), stats=stats)
        try:
            yield gen
        finally:
            gen.close()
```

8. Реалізація власного класу ітератора StudentIterator (streaming.py). Клас інкапсулює протокол ітератора (__iter__ та __next__). Метод __iter__ повертає self. Метод __next__ делегує отримання наступного елемента внутрішньому джерелу, а при вичерпанні фіксує прапорець _exhausted = True та збуджує StopIteration, гарантуючи коректну поведінку вичерпаного ітератора.

```python
class StudentIterator:
    def __init__(self, iterable: Iterable[Student]) -> None:
        # огортаємо ітероване джерело ліниво
        self._source = iter(iterable)
        self._exhausted = False

    def __iter__(self) -> "StudentIterator":
        return self

    def __next__(self) -> Student:
        if self._exhausted:
            raise StopIteration
        try:
            return next(self._source)
        except StopIteration:
            self._exhausted = True
            raise
```

9. Лінива фільтрація, ранній вихід та трансформація даних. Функції filter_by_group та filter_by_min_grade реалізовані як генератори з yield. Функція find_student виконує лінивий пошук за student_id або прізвищем із раннім виходом (return при першому збігу), що дозволяє уникнути читання решти файлу. Функція transform_to_summary ліниво трансформує студентів у скорочені кортежі.

```python
def filter_by_group(students: Iterable[Student], group: str) -> Iterator[Student]:
    # фільтруємо за групою без урахування регістру та зайвих пробілів
    target = group.strip().casefold()
    for student in students:
        if student.group.strip().casefold() == target:
            yield student


def filter_by_min_grade(
    students: Iterable[Student], min_grade: float
) -> Iterator[Student]:
    # фільтруємо за мінімальним балом
    for student in students:
        if student.average_grade >= min_grade:
            yield student


def filter_students(
    students: Iterable[Student],
    *,
    group: str | None = None,
    min_grade: float | None = None,
) -> Iterator[Student]:
    # комбінована лінива фільтрація
    target_group = group.strip().casefold() if group is not None else None
    for student in students:
        if (
            target_group is not None
            and student.group.strip().casefold() != target_group
        ):
            continue
        if min_grade is not None and student.average_grade < min_grade:
            continue
        yield student
```

```python
def find_student(
    students: Iterable[Student],
    *,
    student_id: int | None = None,
    surname: str | None = None,
) -> Student | None:
    # пошук з раннім виходом шоб не обробляти весь файл
    target_surname = surname.strip().casefold() if surname is not None else None
    for student in students:
        if student_id is not None and student.student_id == student_id:
            return student
        if (
            target_surname is not None
            and student.last_name.strip().casefold() == target_surname
        ):
            return student
    return None


def transform_to_summary(
    students: Iterable[Student],
) -> Iterator[tuple[int | None, str, str, float]]:
    # перетворюємо студентів у короткі кортежі
    for student in students:
        yield (
            student.student_id,
            student.full_name,
            student.group.strip(),
            student.average_grade,
        )
```

10. Однопрохідна потокова статистика та ранжування (streaming.py). Функція calculate_statistics обчислює кількість, середній бал, мінімум, максимум і розподіл за групами в одному проході за O(1) додаткової пам'яті (без матеріалізації списку). Функція top_students знаходить k найкращих студентів через heapq.nlargest за O(k) пам'яті. На відміну від неї, first_n_students повертає перші n записів у порядку надходження через itertools.islice.

```python
def calculate_statistics(students: Iterable[Student]) -> dict[str, Any]:
    # однопрохідний підрахунок статистики без списку в пам'яті
    count = 0
    total = 0.0
    min_grade: float | None = None
    max_grade: float | None = None
    group_counts: dict[str, int] = {}
    canonical_groups: dict[str, str] = {}

    for student in students:
        count += 1
        grade = student.average_grade
        total += grade
        if min_grade is None or grade < min_grade:
            min_grade = grade
        if max_grade is None or grade > max_grade:
            max_grade = grade
        grp = student.group.strip()
        key = grp.casefold()
        if key not in canonical_groups:
            canonical_groups[key] = grp
        display_grp = canonical_groups[key]
        group_counts[display_grp] = group_counts.get(display_grp, 0) + 1

    return {
        "count": count,
        "average": (total / count) if count > 0 else None,
        "min_grade": min_grade,
        "max_grade": max_grade,
        "group_counts": group_counts,
    }
```

```python
def top_students(students: Iterable[Student], k: int) -> list[Student]:
    # знаходимо k найкращих за O(k) пам'яті через heapq
    if k < 0:
        raise ValueError("k must be non-negative")
    if k == 0:
        return []
    return heapq.nlargest(k, students, key=lambda s: s.average_grade)


def first_n_students(students: Iterable[Student], n: int) -> list[Student]:
    # перші n записів у порядку надходження через islice
    if n < 0:
        raise ValueError("n must be non-negative")
    return list(itertools.islice(students, n))
```

11. Пакетування (batching) та розширені засоби itertools. Функція batch_students розбиває потік на пакети фіксованого розміру за допомогою itertools.islice. Функція flatten_batches розгортає пакети назад у плаский потік через yield from batch. Функція chain_student_streams послідовно об'єднує потоки декількох файлів через itertools.chain. Функція grade_differences обчислює різниці сусідніх оцінок генераторним виразом та itertools.pairwise. Функція infinite_counter_slice демонструє безпечне обмеження нескінченного потоку itertools.count через islice.

```python
def batch_students(iterable: Iterable[T], size: int) -> Iterator[list[T]]:
    # генератор пакетів фіксованого розміру
    if size <= 0:
        raise ValueError("Batch size must be positive")
    source = iter(iterable)
    while True:
        batch = list(itertools.islice(source, size))
        if not batch:
            break
        yield batch


def flatten_batches(batches: Iterable[Iterable[T]]) -> Iterator[T]:
    # розгортаємо пакети через yield from
    for batch in batches:
        yield from batch
```

```python
def chain_student_streams(*streams: Iterable[Student]) -> Iterator[Student]:
    # об'єднуємо потоки декількох файлів через itertools.chain
    yield from itertools.chain(*streams)


def grade_differences(students: Iterable[Student]) -> Iterator[float]:
    # обчислюємо різниці сусідніх оцінок генераторним виразом
    grades = (s.average_grade for s in students)
    return (round(curr - prev, 4) for prev, curr in itertools.pairwise(grades))


def infinite_counter_slice(
    start: int = 1, step: int = 1, count_limit: int = 10
) -> list[int]:
    # демонструємо скінченний зріз нескінченної послідовності count через islice
    return list(itertools.islice(itertools.count(start, step), count_limit))
```

12. Демонстраційний модуль lab3_demo.py та практичне виконання конвеєра. Сценарій виконує комплексний прогін усіх операцій варіанта 1 над файлом demo_students.csv (27 записів: 25 валідних, 2 некоректних). Нижче наведено вихідний код частин демонстрації.

```python
# 1. демонстрація власного ітератора та його вичерпання
print("\n--- 1. Iterator exhaustion ---")
with open_student_stream(csv_path) as stream:
    iterator = StudentIterator(stream)
    count_read = 0
    for student in iterator:
        count_read += 1
        if count_read <= 2:
            print(
                f"Read: {student.student_id} {student.full_name} ({student.average_grade})"
            )
    print(f"Total read by iterator: {count_read}")
    try:
        next(iterator)
    except StopIteration:
        print("Next call correctly raised StopIteration (exhausted)")

# 2. фільтрація за групою та мінімальним балом
print("\n--- 2. Filter by group and grade ---")
with open_student_stream(csv_path) as stream:
    filtered = list(filter_by_min_grade(filter_by_group(stream, "ФеП-33"), 85.0))
    print(
        f"Qualifying students in group ФеП-33 with grade >= 85.0: {len(filtered)}"
    )
    for s in filtered[:3]:
        print(f"  {s.student_id}: {s.full_name} - {s.average_grade}")

# 3. лінивий пошук за id та прізвищем з раннім виходом
print("\n--- 3. Lazy search with early termination ---")
with open_student_stream(csv_path) as stream:
    found_by_id = find_student(stream, student_id=2)
    if found_by_id:
        print(f"Found ID 2: {found_by_id.full_name} ({found_by_id.group})")

with open_student_stream(csv_path) as stream:
    found_by_surname = find_student(stream, surname="Шевченко")
    if found_by_surname:
        print(
            f"Found surname Шевченко: ID {found_by_surname.student_id}, {found_by_surname.full_name}"
        )
```

```python
# 4. різниця між першими n записами та top-n за балом
print("\n--- 4. First matches vs Top-N ---")
with open_student_stream(csv_path) as stream:
    first_3 = first_n_students(stream, 3)
    print("First 3 in stream arrival order:")
    for s in first_3:
        print(f"  ID {s.student_id}: {s.full_name} ({s.average_grade})")

with open_student_stream(csv_path) as stream:
    top_3 = top_students(stream, 3)
    print("Top 3 by grade (highest):")
    for s in top_3:
        print(f"  ID {s.student_id}: {s.full_name} ({s.average_grade})")

# 5. однопрохідна потокова статистика та трансформація
print("\n--- 5. Statistics (single pass) ---")
stats_tracker = StreamStats()
with open_student_stream(csv_path, stats=stats_tracker) as stream:
    stats = calculate_statistics(stream)
    print(f"Count: {stats['count']}")
    avg_str = f"{stats['average']:.2f}" if stats["average"] is not None else "None"
    print(f"Average: {avg_str}")
    print(f"Min grade: {stats['min_grade']}")
    print(f"Max grade: {stats['max_grade']}")
    print(f"Groups: {stats['group_counts']}")
    print(f"Invalid records skipped: {stats_tracker.invalid_records}")
```

```python
    print("\n--- Summary tuples (transformation) ---")
    with open_student_stream(csv_path) as stream:
        for item in list(transform_to_summary(stream))[:2]:
            print(f"  Summary tuple: {item}")

    # 6. пакетна обробка
    print("\n--- 6. Batches ---")
    with open_student_stream(csv_path) as stream:
        batch_size = 2
        batch_count = 0
        total_in_batches = 0
        for batch in batch_students(stream, batch_size):
            batch_count += 1
            total_in_batches += len(batch)
        print(
            f"Batch size: {batch_size}, total batches: {batch_count}, records processed: {total_in_batches}"
        )

    # 7. об'єднання декількох файлів через chain
    print("\n--- 7. Chain streams ---")
    with open_student_stream(csv_path) as s1, open_student_stream(csv_path) as s2:
        chained = list(first_n_students(chain_student_streams(s1, s2), 4))
        print(
            f"Chained 2 streams, read first 4 records: {[s.student_id for s in chained]}"
        )

    # 8. pairwise різниці оцінок
    print("\n--- 8. Pairwise grade differences ---")
    with open_student_stream(csv_path) as stream:
        first_few = first_n_students(stream, 4)
        diffs = list(grade_differences(first_few))
        print(f"Pairwise differences of first {len(first_few)} students: {diffs}")

    # 9. скінченний зріз нескінченної послідовності count через islice
    print("\n--- 9. Infinite slice (itertools.count + islice) ---")
    infinite_slice = infinite_counter_slice(start=100, step=10, count_limit=5)
    print(f"Generated bounded slice of count: {infinite_slice}")

    return 0
```

Виконав запуск демонстраційного конвеєра в консолі. На першому етапі StudentIterator вичитав усі записи, після чого виклик next() успішно підняв StopIteration. Фільтрація ФеП-33 із балом >= 85.0 повернула 3 студентів. Лінивий пошук миттєво знайшов ID 2 (Тарас Франко) та прізвище Шевченко (ID 18). Ранжування Top-3 за балом (ID 19: 99.88, ID 21: 99.03, ID 5: 95.30) чітко відрізняється від перших трьох записів у файлі (ID 1: 88.45, ID 2: 68.96, ID 3: 92.60), що доводить коректність роботи алгоритму сортування через heap купу.

```
PS C:\Users\ponti> .\.venv\Scripts\python.exe -m student_manager.lab3_demo
.local_workflow\lab3\demo_students.csv
Dataset path: F:\university\3yr-1\professional_python\.local_workflow\lab3\demo_students.csv

--- 1. Iterator exhaustion ---
Read: 1 Олександр Коваль (88.45)
Read: 2 Тарас Франко (68.96)
Total read by iterator: 25
Next call correctly raised StopIteration (exhausted)

--- 2. Filter by group and grade ---
Qualifying students in group ФеП-33 with grade >= 85.0: 3
  3: Ірина Бондар - 92.6
  19: Ольга Шевченко - 99.88
  24: Іван Шевченко - 89.94

--- 3. Lazy search with early termination ---
Found ID 2: Тарас Франко (ФеП-31)
Found surname Шевченко: ID 18, Ольга Шевченко

--- 4. First matches vs Top-N ---
First 3 in stream arrival order:
  ID 1: Олександр Коваль (88.45)
  ID 2: Тарас Франко (68.96)
  ID 3: Ірина Бондар (92.6)
Top 3 by grade (highest):
  ID 19: Ольга Шевченко (99.88)
  ID 21: Марія Мельник (99.03)
  ID 5: Іван Українка (95.3)
PS C:\Users\ponti> _
```

Продовження виведення демонстраційного конвеєра: однопрохідна статистика зафіксувала середній бал 78.93, min 50.02, max 99.88, розподіл груп та пропуск рівно 2 некоректних рядків. Трансформація сформувала короткі кортежі, батчі розбили потік на 13 пакетів по 2 записи, chain об'єднав 2 потоки, pairwise обчислив динаміку оцінок, а islice обмежив count.

```
PS C:\Users\ponti>
--- 5. Statistics (single pass) ---
Count: 25
Average: 78.93
Min grade: 50.02
Max grade: 99.88
Groups: {'ФеП-31': 8, 'ФеП-33': 6, 'ФеП-34': 9, 'ФеП-32': 2}
Invalid records skipped: 2

--- Summary tuples (transformation) ---
  Summary tuple: (1, 'Олександр Коваль', 'ФеП-31', 88.45)
  Summary tuple: (2, 'Тарас Франко', 'ФеП-31', 68.96)

--- 6. Batches ---
Batch size: 2, total batches: 13, records processed: 25

--- 7. Chain streams ---
Chained 2 streams, read first 4 records: [1, 2, 3, 4]

--- 8. Pairwise grade differences ---
Pairwise differences of first 4 students: [-19.49, 23.64, -14.55]

--- 9. Infinite slice (itertools.count + islice) ---
Generated bounded slice of count: [100, 110, 120, 130, 140]
PS C:\Users\ponti> _
```

13. Експериментальне дослідження швидкодії та пам'яті: Eager vs Lazy Pipeline (streaming_experiments.py). Для замірів часу використано time.perf_counter(), а для пікового виділення динамічної пам'яті інтерпретатора Python (heap allocations) - бібліотеку tracemalloc (саме виділення об'єктів у купі Python, а не загальну фізичну RAM процесу ОС). Для забезпечення повної методологічної чистоти та паритету експерименту обидва конвеєри використовують спільний валідований парсер open_student_stream(path). В eager-варіанті весь потік валідованих об'єктів Student матеріалізується у повний список all_students = list(stream) з подальшою фільтрацією списковим включенням filtered = [s for s in all_students ...]. У lazy-варіанті дані проходять через зв'язані генератори без збереження проміжних колекцій у пам'яті. Час генерації синтетичних файлів суворо виключено з інтервалів вимірювання.

```python
def generate_synthetic_csv(path: Path, count: int, seed: int = 42) -> None:
    # детермінована генерація синтетичних студентів пакетами
    rng = random.Random(seed)
    path.parent.mkdir(parents=True, exist_ok=True)

    chunk_size = 50_000
    with path.open("w", encoding="utf-8-sig", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(list(PRESCRIBED_HEADERS))

        current_id = 1
        while current_id <= count:
            batch_limit = min(current_id + chunk_size, count + 1)
            rows = []
            for idx in range(current_id, batch_limit):
                fn = rng.choice(FIRST_NAMES)
                ln = rng.choice(LAST_NAMES)
                grp = rng.choice(GROUPS)
                grade = round(rng.uniform(50.0, 100.0), 2)
                rows.append([idx, fn, ln, grp, grade])
            writer.writerows(rows)
            current_id = batch_limit
```

```python
def measure_time_to_first_lazy(
    path: Path, target_group: str, min_grade: float
) -> float:
    # вимірюємо час до отримання першого збігу у lazy режимі
    t0 = time.perf_counter()
    with open_student_stream(path) as stream:
        filtered = filter_by_min_grade(filter_by_group(stream, target_group), min_grade)
        _ = next(filtered, None)
    t1 = time.perf_counter()
    return t1 - t0


def measure_time_to_first_eager(
    path: Path, target_group: str, min_grade: float
) -> float:
    # вимірюємо час до отримання першого збігу у eager режимі через спільний парсер
    t0 = time.perf_counter()
    with open_student_stream(path) as stream:
        all_students = list(stream)

    target_group_clean = target_group.strip().casefold()
    matching = [
        s
        for s in all_students
        if s.group.strip().casefold() == target_group_clean
        and s.average_grade >= min_grade
    ]
    _ = matching[0] if matching else None
    t1 = time.perf_counter()
    return t1 - t0
```

```python
def measure_eager_pipeline(
    path: Path, target_group: str = "ФеП-33", min_grade: float = 75.0
) -> dict[str, Any]:
    # повна матеріалізація даних у пам'яті з використанням тих самих примітивів
    ttfr = measure_time_to_first_eager(path, target_group, min_grade)

    tracemalloc.start()
    t0 = time.perf_counter()

    with open_student_stream(path) as stream:
        all_students = list(stream)

    target_group_clean = target_group.strip().casefold()
    filtered = [
        s
        for s in all_students
        if s.group.strip().casefold() == target_group_clean
        and s.average_grade >= min_grade
    ]

    stats = calculate_statistics(filtered)

    t1 = time.perf_counter()
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    return {
        "elapsed_seconds": round(t1 - t0, 5),
        "peak_memory_bytes": peak,
        "time_to_first_result_seconds": round(ttfr, 6),
        "stats": stats,
    }

    t1 = time.perf_counter()
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    return {
        "elapsed_seconds": round(t1 - t0, 5),
        "peak_memory_bytes": peak,
        "time_to_first_result_seconds": round(ttfr, 6),
        "stats": stats,
    }
```

```python
def measure_lazy_pipeline(
    path: Path, target_group: str = "ФеП-33", min_grade: float = 75.0
) -> dict[str, Any]:
    # потокова обробка через генераторний конвеєр
    ttfr = measure_time_to_first_lazy(path, target_group, min_grade)

    tracemalloc.start()
    t0 = time.perf_counter()

    with open_student_stream(path) as stream:
        filtered = filter_by_min_grade(filter_by_group(stream, target_group), min_grade)
        stats = calculate_statistics(filtered)

    t1 = time.perf_counter()
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    return {
        "elapsed_seconds": round(t1 - t0, 5),
        "peak_memory_bytes": peak,
        "time_to_first_result_seconds": round(ttfr, 6),
        "stats": stats,
    }
```

Окремо досліджено ефект раннього завершення (Early Termination) на вибірці 1 000 000 записів. Шуканий запис розміщено на позиції student_id = 500. У лінивому конвеєрі цикл переривається оператором break, тоді як eager варіант змушений завантажити та створити об'єкти для всіх 1 000 000 студентів.

```python
def measure_early_termination(
    path: Path, target_student_id: int = 500
) -> dict[str, Any]:
    # порівняння раннього виходу lazy vs повної матеріалізації eager
    # 1. lazy early termination
    tracemalloc.start()
    t0_lazy = time.perf_counter()
    consumed_lazy = 0
    found_lazy = None

    with open_student_stream(path) as stream:
        for s in stream:
            consumed_lazy += 1
            if s.student_id == target_student_id:
                found_lazy = s
                break

    t1_lazy = time.perf_counter()
    _, peak_lazy = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    # 2. eager full materialization
    tracemalloc.start()
    t0_eager = time.perf_counter()

    with open_student_stream(path) as stream:
        all_students = list(stream)

    consumed_eager = len(all_students)
    matching = [s for s in all_students if s.student_id == target_student_id]
    found_eager = matching[0] if matching else None

    t1_eager = time.perf_counter()
    _, peak_eager = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    return {
        "lazy": {
            "elapsed_seconds": round(t1_lazy - t0_lazy, 6),
            "peak_memory_bytes": peak_lazy,
            "records_consumed": consumed_lazy,
            "found_id": found_lazy.student_id if found_lazy else None,
        },
        "eager": {
            "elapsed_seconds": round(t1_eager - t0_eager, 6),
            "peak_memory_bytes": peak_eager,
            "records_consumed": consumed_eager,
            "found_id": found_eager.student_id if found_eager else None,
        },
    }
```

```python
def run_experiments(
    work_dir: Path,
    sizes: list[int] | None = None,
    run_million: bool = True,
    output_json: Path | None = None,
) -> int:
    if sizes is None:
        sizes = [10_000, 100_000, 500_000]

    work_dir.mkdir(parents=True, exist_ok=True)
    results: dict[str, Any] = {"experiments": []}

    for size in sizes:
        csv_file = work_dir / f"dataset_{size}.csv"
        generate_synthetic_csv(csv_file, count=size, seed=42)

        eager_metric = measure_eager_pipeline(csv_file)
        lazy_metric = measure_lazy_pipeline(csv_file)
        assert eager_metric["stats"] == lazy_metric["stats"], (
            "Eager and lazy stats must match"
        )

        results["experiments"].append(
            {
                "size": size,
                "eager_time_sec": eager_metric["elapsed_seconds"],
                "lazy_time_sec": lazy_metric["elapsed_seconds"],
                "eager_peak_bytes": eager_metric["peak_memory_bytes"],
                "lazy_peak_bytes": lazy_metric["peak_memory_bytes"],
                "eager_time_to_first_sec": eager_metric["time_to_first_result_seconds"],
                "lazy_time_to_first_sec": lazy_metric["time_to_first_result_seconds"],
                "records_matched": lazy_metric["stats"]["count"],
                "average_grade": round(lazy_metric["stats"]["average"], 2)
                if lazy_metric["stats"]["average"]
                else None,
            }
        )

        if csv_file.exists():
            csv_file.unlink()

    if run_million:
        million_file = work_dir / "dataset_1000000.csv"
        generate_synthetic_csv(million_file, count=1_000_000, seed=42)
        early_term = measure_early_termination(million_file, target_student_id=500)
        results["million_records_early_termination"] = early_term
        if million_file.exists():
            million_file.unlink()
```

Виконав повний бенчмарк для вибірок 10 000, 100 000, 500 000 та 1 000 000 записів. Нижче наведено перевірку збереженого підсумкового JSON-звіту експерименту (experiment_results.json).

```
PS C:\Users\ponti> Get-Content .local_workflow\lab3\experiment_results.json
{
  "experiments": [
    {
      "size": 10000,
      "eager_time_sec": 0.41066,
      "lazy_time_sec": 0.36027,
      "eager_peak_bytes": 3780347,
      "lazy_peak_bytes": 71560,
      "eager_time_to_first_sec": 0.111582,
      "lazy_time_to_first_sec": 0.000704,
      "records_matched": 1236,
      "average_grade": 87.7
    },
    {
      "size": 100000,
      "eager_time_sec": 5.1944,
      "lazy_time_sec": 5.06513,
      "eager_peak_bytes": 37580113,
      "lazy_peak_bytes": 71832,
      "eager_time_to_first_sec": 1.366329,
      "lazy_time_to_first_sec": 0.000703,
      "records_matched": 12437,
      "average_grade": 87.45
    },
    {
      "size": 500000,
      "eager_time_sec": 29.05654,
      "lazy_time_sec": 23.68636,
      "eager_peak_bytes": 188058969,
      "lazy_peak_bytes": 71737,
      "eager_time_to_first_sec": 5.972384,
      "lazy_time_to_first_sec": 0.000768,
      "records_matched": 62287,
      "average_grade": 87.53
    }
  ],
  "million_records_early_termination": {
    "lazy": {
      "elapsed_seconds": 0.037148,
      "peak_memory_bytes": 70359,
      "records_consumed": 500,
      "found_id": 500
    },
    "eager": {
      "elapsed_seconds": 65.868735,
      "peak_memory_bytes": 375273898,
      "records_consumed": 1000000,
      "found_id": 500
    }
  }
}
PS C:\Users\ponti> _
```

14. Зведена таблиця експериментальних результатів (10 000, 100 000, 500 000 записів). Дані отримано на основі фактичного виконання streaming_experiments.py. Результати агрегації в обох режимах повністю збігаються (наприклад, для 100k: matched = 12 437 студентів, average = 87.45).

Вибірка (N)	Eager час, с	Lazy час, с	Eager пікова RAM	Lazy пікова RAM	Eager TTFR, с	Lazy TTFR, с
10 000	0.41066	0.36027	3 780 347 Б (3.61 MiB)	71 560 Б (69.88 KiB)	0.11158	0.000704 (0.70 мс)

100 000	5.19440	5.06513	37 580 113 Б (35.84 MiB)	71 832 Б (70.15 KiB)	1.36633	0.000703 (0.70 мс)
500 000	29.05654	23.68636	188 058 969 Б (179.35 MiB)	71 737 Б (70.06 KiB)	5.97238	0.000768 (0.77 мс)

15. Зведена таблиця дослідження раннього завершення на 1 000 000 записів (Early Termination). Пошук студента з student_id = 500 демонструє колосальну перевагу потокового підходу.

Режим обробки	**Спожито рядків**	**Час виконання, с**	**Пікова пам'ять (RAM)**	**Знайдений ID**	**Характеристика ефекту**
Lazy pipeline (ранній вихід)	500	0.03715	70 359 Б (68.71 KiB)	500	Зупинка на 500-му рядку, решта 999 500 рядків не читалися.
Eager матеріалізація (повний список)	1 000 000	65.86874	375 273 898 Б (357.89 MiB)	500	У 1773х повільніше; у 5334х більше пам'яті під 1M об'єктів.

16. Аналіз результатів вимірювань та порівняння eager і lazy підходів:
- Пікова пам'ять (Peak Memory за tracemalloc): Лінивий конвеєр демонструє практично константне споживання пам'яті в купі інтерпретатора близько 70 КіБ незалежно від масштабу вибірки (69.88 КіБ для 10k, 70.15 КіБ для 100k, 70.06 КіБ для 500k та 68.71 КіБ для 1M). Натомість в eager-варіанті виділення пам'яті під об'єкти зростає строго лінійно O(n): від 3.61 МіБ для 10k (3 780 347 Б) до 35.84 МіБ для 100k (37 580 113 Б), 179.35 МіБ для 500k (188 058 969 Б) та 357.89 МіБ для 1M (375 273 898 Б). Для 500 000 записів лінивий конвеєр вимагає у 2622 разів менше динамічної пам'яті інтерпретатора;
- Час до першого результату (Time-to-First-Result, TTFR): У lazy-режимі споживач отримує перший валідний запис практично миттєво (0.70 мс для 10k, 0.70 мс для 100k та 0.77 мс для 500k) незалежно від загального розміру файлу. В eager-режимі TTFR вимагає попередньої повної матеріалізації всього вхідного потоку і становить 111.6 мс (0.112 с) для 10k, 1.37 с для 100k та 5.97 с для 500k;
- Загальний час виконання (Total Elapsed Time): Обидва конвеєри використовують однаковий валідований парсер, тому час обробки на вибірках 10k та 100k є порівнянним (0.360 с у lazy проти 0.411 с в eager для 10k; 5.07 с проти 5.19 с для 100k). На вибірці 500k потоковий конвеєр випереджає eager (23.69 с проти 29.06 с) через відсутність накладних витрат на алокацію та збирання великих масивів об'єктів у пам'яті;
- Ранній вихід (Early Termination): На вибірці 1 000 000 записів лінивий пошук спрацював за 0.03715 секунди та 68.71 КіБ пам'яті (70 359 Б), тоді як eager

матеріалізація зайняла 65.87 секунди та 357.89 МіБ (375 273 898 Б). Прискорення склало 1773х, а скорочення пікової пам'яті в купі - 5334х.

17. Реалізація завдань підвищеної складності (>=3 додаткових розширень). У лабораторній роботі реалізовано наступні розширені можливості з інструкції:
1) Власний клас ітератора StudentIterator: реалізує протокол __iter__ / __next__, коректно піднімає StopIteration та відстежує стан вичерпання;
2) Потокове об'єднання декількох файлів chain_student_streams: використовує itertools.chain для безперервної конкатенації потоків без матеріалізації проміжних списків;
3) Відстеження та підрахунок некоректних рядків: реалізовано структури StreamStats та invalid_counter, що фіксують відкинуті рядки без переривання конвеєра;
4) Вимірювання часу до першого результату (TTFR): окрема методологія заміру відгуку в streaming_experiments.py;
5) Лінивий пошук із раннім виходом та безпечним закриттям ресурсів: функція find_student та менеджер open_student_stream із гарантованим gen.close() при break.

18. Автоматичне модульне тестування (unittest). Для підтвердження надійності розроблено набір із 27 тестів у тестових модулях test_streaming.py та test_streaming_experiments.py. Тести перевіряють: розширення моделі Student із student_id, лініву природу parse_students (обчислення індексів лише при next()), коректне дострокове закриття дескриптора файлу без помилок PermissionError, роботу heapq Top-N, пакетну обробку, а також строгий збіг розрахованих показників між eager та lazy конвеєрами на тестових вибірках.

```
    def test_parse_students_is_lazy(self) -> None:
        evaluated_indices = []

        def row_generator():
            for i in range(10):
                evaluated_indices.append(i)
                yield [str(i + 1), f"First{i}", f"Last{i}", "ФеП-33", "80.0"]

        stream = parse_students(row_generator())
        self.assertEqual(evaluated_indices, [])  # nothing evaluated yet
        first = next(stream)
        self.assertEqual(evaluated_indices, [0])
        self.assertEqual(first.student_id, 1)

    def test_open_student_stream_context_manager_early_closure(self) -> None:
        with tempfile.NamedTemporaryFile("w", encoding="utf-8-sig", delete=False) as f:
            f.write("student_id,first_name,last_name,group,average_grade\n")
            f.write("1,Тарас,Шевченко,ФеП-33,95.0\n")
            f.write("2,Іван,Франко,ФеП-33,90.0\n")
            path = Path(f.name)

        try:
            with open_student_stream(path) as stream:
                first = next(stream)
                self.assertEqual(first.student_id, 1)
            # після виходу з with файл закритий і може бути видалений
            path.unlink()
        finally:
            if path.exists():
                path.unlink()
```

```python
    def test_eager_vs_lazy_pipeline_results_match(self) -> None:
        csv_path = self.work_dir / "test_500.csv"
        generate_synthetic_csv(csv_path, count=500, seed=42)

        eager_res = measure_eager_pipeline(
            csv_path, target_group="ФеП-33", min_grade=70.0
        )
        lazy_res = measure_lazy_pipeline(
            csv_path, target_group="ФеП-33", min_grade=70.0
        )

        # результати агрегації мають бути ідентичними
        self.assertEqual(eager_res["stats"]["count"], lazy_res["stats"]["count"])
        self.assertAlmostEqual(
            eager_res["stats"]["average"], lazy_res["stats"]["average"], places=4
        )
        self.assertEqual(
            eager_res["stats"]["min_grade"], lazy_res["stats"]["min_grade"]
        )
        self.assertEqual(
            eager_res["stats"]["max_grade"], lazy_res["stats"]["max_grade"]
        )
        self.assertEqual(
            eager_res["stats"]["group_counts"], lazy_res["stats"]["group_counts"]
        )

        # пам'ять у lazy має бути суттєво меншою
        self.assertLess(lazy_res["peak_memory_bytes"], eager_res["peak_memory_bytes"])
```

Результат запуску повного набору модульних перевірок: усі 27 тестів пройдено успішно.

```
PS C:\Users\ponti> .\.venv\Scripts\python.exe -m unittest tests.test_streaming
tests.test_streaming_experiments
...........................
----------------------------------------------------------------------
Ran 27 tests in 0.245s

OK
PS C:\Users\ponti> _
```

19. Перевірка компіляції та синтаксису файлів конвеєра. Виконав компіляцію байткоду модулів streaming.py, streaming_experiments.py та lab3_demo.py за допомогою стандартної утиліти py_compile. Синтаксичних помилок чи зауважень інтерпретатора не виявлено.

```
PS C:\Users\ponti> .\.venv\Scripts\python.exe -m py_compile src\student_manager\streaming.py
src\student_manager\streaming_experiments.py src\student_manager\lab3_demo.py
Синтаксичних помилок не виявлено. Компіляція байткоду успішна.
PS C:\Users\ponti> _
```

Контрольні запитання

1. erator) is iterator?

(У завданні формулювання запитання усічене: 'erator) is iterator?', мається на увазі перевірка тотожності iter(iterator) is iterator). В ітераційній моделі Python для будь-якого ітератора виклик iter(iterator) повертає сам цей об'єкт ітератора (тобто iter(it) is it повертає True), оскільки метод __iter__() ітератора повертає self.

2. Чи можна повторно використати exhausted generator?

Ні, вичерпаний генератор є одноразовим об'єктом (one-shot), і повторна ітерація по ньому одразу піднімає StopIteration або повертає порожню послідовність.

Для повторної обробки потрібно створити новий екземпляр генератора або заново викликати генераторну функцію.

3. Що таке generator function?

Це функція, яка містить ключове слово yield замість звичайного return. При виклику вона не виконує тіло одразу, а повертає спеціальний об'єкт генератора (generator iterator), який генерує значення за вимогою під час викликів next().

4. Яку роль виконує yield?

Інструкція yield повертає чергове обчислене значення споживачу та тимчасово призупиняє виконання функції. При цьому повністю зберігається стан локальних змінних, покажчик поточної інструкції та контекст для продовження при наступному виклику next().

5. Чим yield відрізняється від return?

Інструкція return остаточно завершує виконання функції та знищує її стек виклику. На відміну від неї, yield призупиняє виконання зі збереженням локального стану функції, дозволяючи відновлювати роботу з тієї самої точки при наступних запитах.

6. Коли фактично починає виконуватися generator function?

Тіло генераторної функції починає фактично виконуватися лише під час першого виклику функції next() над отриманим об'єктом генератора (або на початку циклу for). Сам початковий виклик функції лише інстанціює об'єкт генератора без виконання коду.

7. Що зберігає generator між викликами next()?

Генератор зберігає повний стан кадру виконання (frame object): значення всіх локальних змінних, стан циклів і блоків try/finally, а також індекс поточної інструкції байткоду (instruction pointer).

8. Що таке generator expression?

Це компактний синтаксичний вираз у круглих дужках, аналогічний list comprehension, який створює анонімний об'єкт генератора. Він обчислює кожен елемент послідовності ліниво за вимогою без виділення пам'яті під усю колекцію.

9. Чим generator expression відрізняється від list comprehension?

List comprehension матеріалізує всі елементи у новий список у пам'яті одразу під час виконання виразу. Generator expression повертає лінивий ітератор, який генерує елементи по одному за потреби з мінімальним константним споживанням пам'яті.

10. Що таке lazy evaluation?

Це стратегія обчислень, за якої значення виразу обчислюється лише в той момент, коли воно безпосередньо стає потрібним споживачеві. Це запобігає зайвим обчисленням та мінімізує використання оперативної пам'яті.

11. Що таке eager evaluation?

Це стратегія енергійних (жадібних) обчислень, за якої всі елементи та вирази обчислюються й зберігаються в пам'яті повністю та негайно при виклику операції.

12. Які переваги lazy evaluation?

Мінімальне та константне використання оперативної пам'яті, можливість обробки потенційно нескінченних послідовностей і великих файлів, суттєве зменшення часу до першого результату (TTFR) та ефективна підтримка раннього виходу (early termination).

13. Які недоліки lazy evaluation?

Одноразовість ітераторів (неможливість повторного проходження без перезапуску), відсутність прямого доступу за індексом O(1) чи визначення len(), а також відкладене виявлення винятків (exceptions виникають лише під час споживання, а не конструювання).

14. Що таке streaming?

Це метод обробки даних частинами (записами або фіксованими пакетами) у міру їх надходження з джерела, без повного одночасного завантаження всього набору даних у RAM.

15. Чому streaming важливий для великих files?

Він дозволяє обробляти файли, обсяг яких значно перевищує розмір доступної оперативної пам'яті комп'ютера, запобігаючи помилкам MemoryError та виснаженню системних ресурсів.

16. Чому read() може бути проблемою для multi-GB file?

Метод read() без аргументів намагається завантажити весь вміст файлу в єдиний монолітний рядок або буфер пам'яті. Для багатогігабайтних файлів це призводить до аварійного вичерпання RAM (MemoryError) або зависання системи через активний swap.

17. Чому file object є iterable?

Об'єкт файлу реалізує протокол ітератора: він має методи __iter__() та __next__(), які повертають рядки файлу по одному за викликом буферизованого системного читання.

18. Для чого використовується yield from?

Конструкція yield from делегує ітерацію підітератору або вкладеному ітерованому об'єкту, автоматично пересилаючи всі елементи, значення, винятки

та виклики send/throw/close між зовнішнім споживачем і вкладеним генератором.

19. Що таке itertools?

Це стандартний модуль Python, що містить високоефективні швидкі будівельні блоки на мові C для створення, комбінування та трансформації лінивих ітераторів.

20. Для чого itertools.count()?

Функція itertools.count(start, step) створює нескінченний числовий ітератор, який послідовно генерує значення починаючи зі start з кроком step.

21. Чому count() потрібно обмежувати?

Оскільки count() є нескінченним генератором, спроба його повної матеріалізації (наприклад, через list(count())) призведе до нескінченного циклу та вичерпання оперативної пам'яті. Його споживання необхідно обмежувати за допомогою itertools.islice або умови break.

22. Для чого islice()?

Функція itertools.islice(iterable, [start], stop, [step]) виконує лінивий зріз будь-якого ітератора без створення проміжних списків у пам'яті, що критично для обмеження нескінченних потоків.

23. Для чого chain()?

Функція itertools.chain(*iterables) послідовно об'єднує декілька ітерованих об'єктів в один безперервний потік, починаючи ітерацію по наступному об'єкту лише після повного вичерпання попереднього.

24. Як працює takewhile()?

Функція itertools.takewhile(predicate, iterable) повертає елементи з вхідного потоку доти, доки предикат повертає True. Як тільки предикат стає False, ітератор негайно припиняє роботу й ігнорує всі наступні елементи.

25. Як працює dropwhile()?

Функція itertools.dropwhile(predicate, iterable) пропускає всі початкові елементи потоку, поки предикат повертає True. Після першого ж елемента, для якого предикат дав False, вона повертає цей елемент і всі наступні елементи потоку без перевірки.

26. Для чого accumulate()?

Функція itertools.accumulate(iterable, [func]) генерує потік накопичуваних (кумулятивних) проміжних результатів застосування бінарної функції (за замовчуванням операції додавання) до елементів послідовності.

27. Для чого pairwise()?

Функція itertools.pairwise(iterable) повертає послідовні пари сусідніх елементів потоку, що перекриваються: (s0, s1), (s1, s2), (s2, s3). Вона зручна для розрахунку різниць сусідніх значень, динаміки та аналізу часових рядів.

28. Як працює groupby()?

Функція itertools.groupby(iterable, key=None) об'єднує послідовні сусідні елементи з однаковим значенням ключа у підгрупи, повертаючи пари (ключ, ітератор_групи).

29. Чому перед groupby() часто потрібне sorting?

groupby() розбиває елементи на нову групу щоразу, коли змінюється значення ключа. Якщо елементи з однаковим ключем розкидані по колекції, вони утворять окремі фрагментовані групи; сортування за тим самим ключем гарантує об'єднання всіх однакових записів.

30. Що робить product()?

Функція itertools.product(*iterables, repeat=1) обчислює декартовий добуток вхідних послідовностей, що еквівалентно вкладеним циклам for у лінивому виконанні.

31. Чим combinations() відрізняється від permutations()?

permutations(r) генерує всі можливі перестановки елементів довжини r, де порядок елементів має значення. combinations(r) генерує лише унікальні невпорядковані комбінації, де кортежі з однаковими елементами в різному порядку вважаються дублікатами й не повторюються.

32. Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?

Кількість комбінацій зростає експоненціально або факторіально (комбінаторний вибух). Навіть за лінивого виділення пам'яті для окремого елемента, повний перебір такої кількості варіантів потребує гігантських часових витрат процесора.

33. Що таке batching?

Це розбиття неперервного потоку окремих елементів на порції (пакети, батчі) фіксованого або адаптивного розміру для групової обробки.

34. Які переваги batching?

Зменшення накладних витрат на системні виклики та операції введення-виведення, оптимізація мережевих запитів до API, пакетні вставки в бази даних (bulk insert) та збалансоване використання пам'яті.

35. Що таке lazy pipeline?

Це архітектурний шаблон конвеєра обробки даних, у якому декілька генераторних функцій з'єднані послідовно (вихід одного генератора

передається на вхід наступного), а дані просуваються конвеєром по одному елементу лише за запитом кінцевого споживача.

36. Які stages може містити pipeline?

Типовий конвеєр містить етапи: джерело (source/reader), сирий синтаксичний аналіз (parser), валідація та очищення (validation/cleaning), фільтрація (filtering), трансформація (transformation), пакетування (batching) та фінальна агрегація/збереження (aggregation/sink).

37. Що є source у streaming pipeline?

Це початковий компонент (генератор або ітератор), який читає необроблені дані із зовнішнього джерела (файлу, сокета, черги повідомлень або генератора випадкових чисел) та передає їх у потік.

38. Що є transformation?

Це проміжний етап конвеєра, який приймає вхідні об'єкти та перетворює їх на інші типи або структури (наприклад, перетворення рядків на екземпляри датакласів чи формування скорочених кортежів).

39. Що є sink?

Це кінцевий споживач конвеєра (термінальна операція), який ітерує генератор і матеріалізує результат у файл, відправляє в базу даних або згортає в єдине скалярне значення чи словник статистики.

40. Чи гарантує generator менший total execution time?

Ні, генератор не гарантує менший загальний час виконання, оскільки перемикання контексту, призупинення та відновлення кадрів генератора несуть додаткові накладні витрати інтерпретатора. Проте генератор суттєво зменшує time to first result і пікове споживання пам'яті.

41. Яка основна memory advantage generator?

Постійне (константне O(1)) споживання оперативної пам'яті, яке не залежить від загальної кількості записів у наборі даних, оскільки в пам'яті одночасно знаходиться лише один поточний оброблюваний елемент.

42. Що таке time to first result?

Це інтервал часу від моменту ініціалізації операції до моменту отримання та готовності до використання найпершого результуючого елемента споживачем.

43. Чому lazy pipeline добре підтримує early termination?

Оскільки елементи обчислюються по одному за вимогою, конвеєр можна зупинити (наприклад, через break при знаходженні потрібного запису), не витрачаючи час процесора та пам'ять на читання й обробку решти набору даних.

44. Чому exception у generator може виникати із затримкою?

Код генераторної функції виконується лише в момент виклику next() над генератором; тому помилка у виразах або даних виникає лише під час реального споживання відповідного проблемного елемента, а не на етапі конструювання генератора.

45. Чому list(generator) може бути проблемою?

Виклик list() примусово й негайно матеріалізує всі елементи генератора в оперативну пам'ять, що повністю нівелює переваги лінивої обробки та може спричинити помилку MemoryError на великих наборах даних.

46. Що трапиться при list(count())?

Виникне нескінченний цикл, інтерпретатор безперервно виділятиме пам'ять під зростаючий список, і програма зависне або аварійно завершиться через вичерпання доступної RAM (MemoryError).

47. Як generator може впливати на lifecycle file resource?

Якщо генератор відкриває файл у менеджері контексту with і призупиняється через yield, дескриптор файлу залишається відкритим в ОС до повного вичерпання або закриття генератора. При достроковому виході генератор необхідно явно закривати (через generator.close() або contextmanager), щоб не виникало блокування файлу.

48. Як тестувати невеликий generator?

Невеликий генератор тестують матеріалізацією через list() або tuple() із подальшим порівнянням отриманого списку з еталоном (self.assertEqual(list(gen), expected)), або послідовними викликами next() із перевіркою StopIteration.

49. Як тестувати infinite generator?

Нескінченний генератор тестують за допомогою itertools.islice, вибираючи фіксовану скінченну кількість елементів (наприклад, list(itertools.islice(gen, N))), або через окремі точкові виклики next().

50. Коли краще використовувати list, а коли generator?

List краще використовувати, коли потрібен багаторазовий обхід, доступ за довільним індексом O(1), сортування in-place або знання повної довжини len(). Generator слід обирати для великих або нескінченних наборів даних, потокового читання файлів, довгих конвеєрів трансформацій та задач із можливим раннім виходом.

Висновок:

У ході виконання лабораторної роботи №3 розробив та верифікував повнофункціональний конвеєр потокової обробки даних варіанта 1 на основі зв'язаних генераторів, власного ітератора StudentIterator та інструментів itertools. Обидва шляхи експериментального порівняння реалізовано через спільний валідований парсер open_student_stream, а вимірювання пікової пам'яті виконано через бібліотеку tracemalloc. Експериментально дослідив відмінності між eager та lazy підходами на вибірках до 1 000 000 записів: лінивий конвеєр забезпечує стабільне константне використання динамічної пам'яті близько 70.1 КіБ незалежно від обсягу даних (економія пам'яті при ранньому виході у 5334 разів), миттєвий відгук до першого результату (~0.77 мс проти 5.97 с на 500k) та колосальне прискорення раннього завершення в 1773 разів. Встановив, що завдяки усуненню масивних алокацій у пам'яті потоковий підхід демонструє перевагу у швидкодії на великих наборах і дозволяє обробляти великі файли без повної матеріалізації в пам'яті.